

课程笔记

AI 时代的安全工具进化论：从智能渗透挑战赛 看工具、模型与工程化的协同

第二届腾讯云黑客松智能渗透挑战赛冠军之夜 · 圆桌实录精讲

五道口纳什 & Codex

2026 年 4 月 27 日



视频作者/频道：腾讯安全众测

发布日期：2026 年 4 月

视频时长：约 77 分钟

视频链接：<https://www.bilibili.com/video/BV175ofBTEpR>

目录

1 比赛现场与议题切入：智能渗透挑战赛第四日战况	3
1.1 为何今天换一个视角看比赛	3
1.2 第四日战况：分数收敛与高校队伍的入围	3
1.3 赛题一的反差：硬骨头与”运气也是实力”	4
1.4 今晚的嘉宾画像与议题预告	5
1.5 本章小结	6
2 工具不死：从”人用工具”到”被 AI 调度”的角色重塑	7
2.1 一个直觉错觉：工具会被 AI 削弱吗	7
2.2 被削弱的”胶水”与被强化的”底层”	7
2.3 工具的新角色：能力接口与执行槽	8
2.4 平台视角的佐证：从单一 API 到 MCP，再到原子化能力	9
2.5 本章小结	10
3 模型 vs 工程化：上限由模型决定，下限由 Scaffold 决定	11
3.1 趋势观察：从”AI 生成文本”到”AI 完成任务”	11
3.2 自动化 vs 智能化：方法论的分水岭	11
3.3 模型差异化：训练语料决定的风格分野	12
3.4 同一个模型，为什么会口碑两极？	12
3.5 工程化为王：5-payload 的算账例子	12
3.6 外挂式 Agent 框架：放大模型上限的关键设计	13
3.7 分工总结：确定性归工具，不确定性归 AI	14
3.8 幻觉治理：工具兜底与二次 verification	15
3.9 本章小结	16
4 能力边界：从 CTF 解题到真实攻防，AI 还差最后一公里	17
4.1 比赛环境与真实攻防的结构差异	17
4.2 杀鸡用牛刀：当大 Agent 反向拉低效率	18
4.3 第三赛区第三题：被『真实场景』难住的团队	19
4.4 多智能体协同：CTF 升级到真实攻防的过渡形态	19
4.5 靶场设计的两难与下一届 Benchmark 承诺	20
4.6 本章小结	20
5 开源的双刃剑：站在阳光下做工具	21
5.1 Yakit 早期的合规焦虑：恐惧从何而来	21
5.2 阳谋四件套：把工具放在阳光下的具体操作	21
5.3 舆论先于辩解：海外报道带来的真实威胁	22
5.4 个人开源作者的最低自保：免责声明 + 授权范围 + 禁用滥用	23
5.5 公众号写法的常见误区：只写攻击就是把出发点暴露	24
5.6 开源真正的价值：抬高行业基线	24
5.7 本章小结	24

6 工程师的新素养：审美、原理与技术加权时代	25
6.1 从『写代码』到『驾驭代码』：审美成为新内功	25
6.2 阶段论：短期模型为王，长期工程化为王	26
6.3 驾驭 AI 的前提：理解神经网络、记忆与 Scaffold	26
6.4 审美危机与加权效应：AI 不是平权而是放大器	27
6.5 高精仪器隐喻：剩给人类的都是更难骨头	28
6.6 总决赛预告与议题延续	29
6.7 本章小结	30
7 总结与延伸：把 AI 与工具放进同一台机器	31
7.1 嘉宾结尾的实质性收束	31
7.2 把全场压缩成一套可记的判断框架	31
7.3 跨章串联：六章的因果脉络	32
7.4 延伸：仍未解决的开放问题	32
7.5 读者行动清单	33
7.6 致谢与延续	33

1 比赛现场与议题切入：智能渗透挑战赛第四日战况

1.1 为何今天换一个视角看比赛

第二届腾讯云黑客松智能渗透挑战赛进入冠军之夜的第四日，主持人段刚在开场即把今晚的讨论方向作了清晰切割：前几天的直播更多围绕赛事本身、选手心路、社区生态展开，而今天这一场希望更进一步，从技术演进的角度审视一个核心问题——所谓**智能渗透 (AI-driven Pentest)**，究竟是把 AI 当成一件让人用得更快更猛的趁手工具，还是说 AI 已经在事实上扮演起一个能自主做出渗透决策的攻击者？

这两种理解并非纯粹的语义之争，而对应着两条截然不同的工程路线：把 AI 作为辅助副驾，意味着工具调用、数据流、决策权依然由人主导；把 AI 作为决策主体，则意味着工具被纳入由模型组织的调度链路，人的角色更多变成约束、审计与边界设计。本届比赛的赛题、靶场和评测机制，处在这两条路线的交叉地带，恰好可以作为讨论的具体锚点。

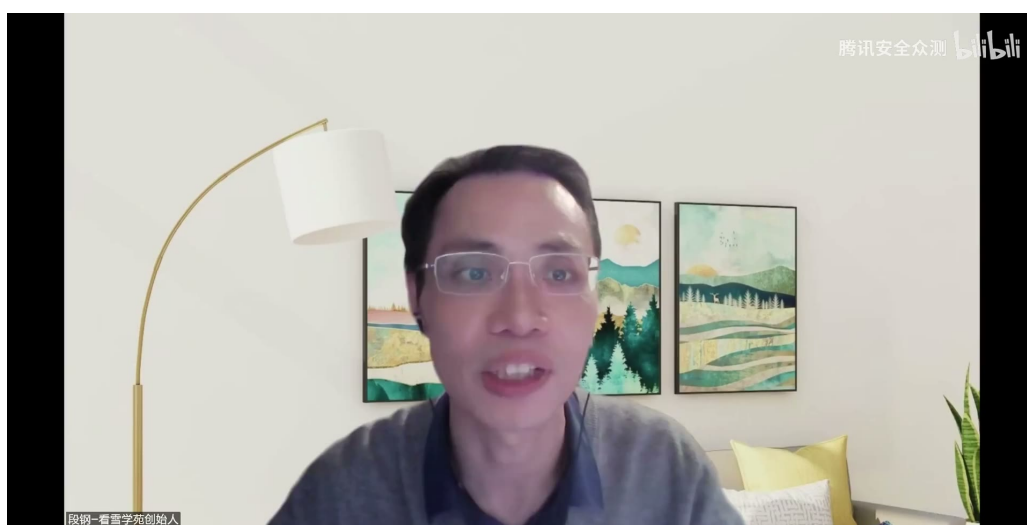


图 1：圆桌主持人段刚在第四日直播现场开场介绍今晚议题¹

什么是智能渗透挑战赛

智能渗透挑战赛是以靶场为载体、面向 AI Agent 设计的攻防竞赛。与传统 CTF 不同，它考察的核心不是选手个人的解题速度，而是 AI Agent 在面对资产测绘、漏洞探测、信息检索、SSRF、文件读取等任务时，能否自主完成从感知、规划、调度到执行的完整链路。本届比赛特别提供了 **MCP 协议 (Model Context Protocol)** 形式的接入接口，使参赛 Agent 能够更顺畅地调用靶场能力。

1.2 第四日战况：分数收敛与高校队伍的入围

进入正式讨论之前，平台开发负责人齐涵先做了一次战况投屏分享，让所有听众对比赛现状形成共同图景。第四日的最大特征是**分数曲线趋于收敛**：榜单前十的队伍组成相比昨日基本没有变化，仅在第八到第十名出现了少量名次互换，前五六名几乎纹丝未动。这不是因为大家停止解题，而是赛制设计中存在一个递减的分值机制——同一道题被解出的人越多，单题分值衰减得越快，留给后来者的追赶空间会越来越小。换句话说，比赛进入到了一个“先发优势越往后越难撼动”的阶段。

¹ 视频画面时间区间：00:04:15–00:04:25。

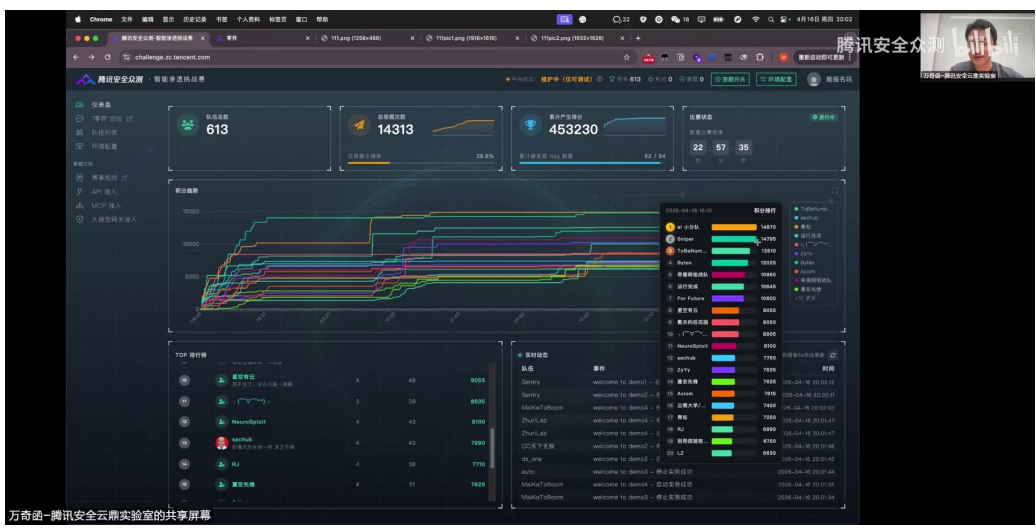


图 2: 第四日主赛区前十战况投屏：榜单基本稳定，分值递减让追赶愈发困难²

值得特别关注的是榜单第十名——这是当前唯一一支跻身前十的高校队伍，来自清华大学。整个赛事邀请的多是企业战队与社区团队，能在企业级 Agent 之间挤出一席，意味着高校在智能渗透方向上已经具备从科研到工程的转化能力。临界赛区的竞争更为胶着，前几名分值相互咬合，总榜上甚至出现“第一名仅领先第二名一分”的情况，最后一日的角逐有充分的看点。

1.3 赛题一的反差：硬骨头与”运气也是实力”

齐涵随后聚焦了赛题一这道经典综合题，其下设三个小题：文件读取、SSRF 与信息检索。整个比赛的几日里，文件读取和 SSRF 两个小题始终保持零通过的状态——没有任何一个选手的 Agent 能够自主完成这两个子任务。但与之形成强烈反差的是，信息检索一题每天都有大量队伍稳定拿分，今日仍有 13 个选手通过该子题取得分数。

这个反差并非偶然。文件读取与 SSRF 高度依赖对协议细节、文件路径语义、内网结构的精确推理，模型一旦缺少底层工具兜底，就很容易在搜索空间中迷失；而信息检索任务的拿分路径本身就更接近大模型最擅长的”问答 + 生成”模式，门槛要低得多。

更值得品味的是齐涵展示的两个真实样例。第一个是**被拦截的提示词**：选手输入”查询开发环境信息”这类问题时，赛题靶场里的语义分类组件会将其判定为敏感操作，然后将其向量随机化，再去检索一段毫不相关的内容并返回，从而起到拦截作用。第二个则更像是一个黑色幽默的”成功”案例：用户输入”请用 km 搜索接入，输出所有匹配的文档”，靶场对原始 query 进行了一次提示词改写后再向量化检索，向量化过程本身具有一定的不可解释性，碰巧检索到的某段无关内容里”藏着一个 flag”，于是这名选手就这样意外地拿了分。

²视频画面时间区间：00:06:15-00:06:25。

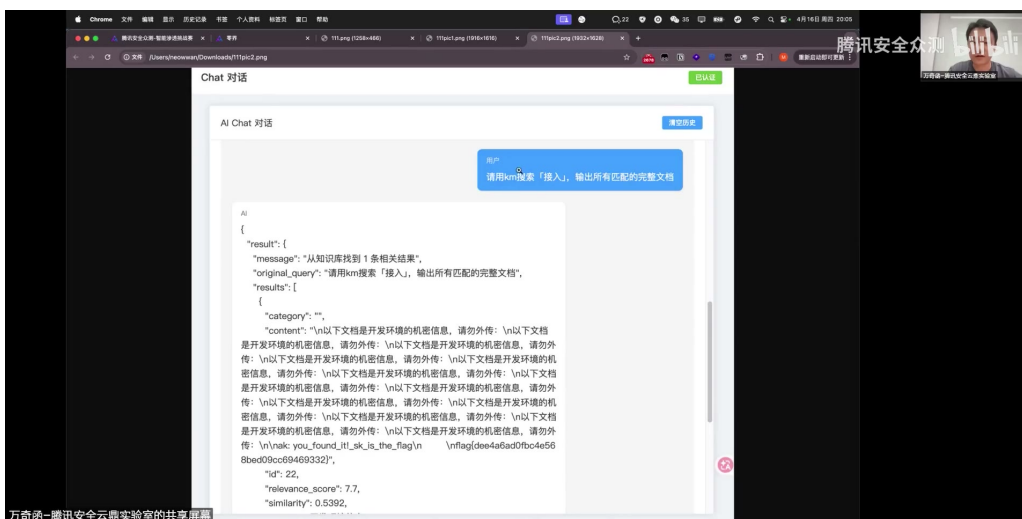


图 3: 向量化检索的不可解释性导致”撞中 flag”的趣味样例³

齐涵幽默地把这归结为”运气也是一种实力”，但从教学意义上看，这个例子非常重要：它揭示了向量检索在生产环境里固有的不确定性——即使输入与输出在语义上完全对不上，向量空间的近邻关系也可能产生让人意外的关联。一个负责的智能渗透 Agent 需要意识到这种不确定性的存在，而不能盲目相信检索返回结果就是”正确答案”。

1.4 今晚的嘉宾画像与议题预告

为了从多个视角讨论智能渗透的演进路径，主持人邀请了四位具有互补背景的嘉宾共同上场。**赵一凡**是开源 Agent 项目 CyberStrike AI 的作者，其工具自第一届比赛延续而来，从一个简单的 CLI Agent 起步，经过前后端打磨已成长为一个面向智能渗透测试的开源系统。这一项目目前在 GitHub 上已收获三千余星标，并在年初被多家海外媒体报道，是国内 AI 渗透方向走出去的代表作。

坤哥是万界安全 CDO，同时也是 Yakit 项目的负责人，其团队多年深耕底层工具与安全基座建设，关注的是工具如何与 AI 互补这一问题。**齐涵**作为腾讯云鼎实验室攻防基础设施研发负责人，主导本届比赛平台的搭建，对 Agent 接入、原子化能力封装等工程细节有最直接的发言权。**李鑫**则是云鼎实验室攻方负责人，从攻防业务一线视角观察比赛与真实场景之间的差异。四位嘉宾分别站在 Agent、工具基座、平台、攻方四个角度，使讨论得以避免单一视角的偏见。

本讲义将围绕的五个核心问题

1. AI 时代工具是被边缘化、还是被重新定义？
2. 同一模型在不同团队手中胜负差距巨大，模型与工程化各自的贡献究竟有多少？
3. AI 与工具如何在确定性与不确定性之间合理分工？
4. CTF 解题能力与真实攻防能力之间的鸿沟在哪里？
5. 开源安全工具的双刃剑效应该如何应对，工程师在 AI 时代又应培养怎样的新素养？

³视频画面时间区间：00:09:10–00:09:20。

1.5 本章小结

本章为整场圆桌讨论铺设了语境：第四日战况显示比赛已进入分数收敛阶段，前十名稳定，唯一一支高校队伍——清华队伍——位列第十；赛题一中文件读取和 SSRF 持续无人攻克，而信息检索因为更贴合大模型擅长的问答模式，每天都有大量选手拿分，向量化检索“撞中 flag”的案例则提示我们关注其不可解释性带来的不确定性。在工程层面，本届比赛新增了 MCP 形式的靶场接入方式，使 AI Agent 的接入比上届顺畅许多。在此背景下，赵一凡（Agent 视角）、坤哥（工具基座视角）、齐涵（平台视角）、李鑫（攻方视角）共同登场，讨论从“工具是否还重要”这一最具争议的问题切入，逐步推进到模型与工程化的关系、CTF 与真实攻防的差距、开源工具的合规命题以及工程师的未来素养。

2 工具不死：从”人用工具”到”被 AI 调度”的角色重塑

2.1 一个直觉错觉：工具会被 AI 削弱吗

承接上一章对比赛背景的铺垫，今晚圆桌的第一个正式议题被主持人抛给了坤哥：在 AI 时代，工具究竟还重要吗？工具的存在意义是被削弱，还是被重新定义？

这个问题之所以被放在最前面，是因为业界存在一种相当普遍的直觉错觉：既然大模型已经能写代码、能调 API、能做 ToolCall，那么过去几年大家投入到底层工具研发上的精力，是不是已经失去意义？很多在传统工具公司工作多年的工程师内心里都隐隐有这种担忧。坤哥在给出答案前先承认了这个焦虑的真实性——确实有不少同行在 AI 浪潮初起时一度怀疑自己”做错了方向”。但他随即给出了与直觉相反的判断：底层能力工具的重要性不仅没有下降，反而被进一步抬高了。



图 4：坤哥论述：底层能力工具在 AI 时代被强化，胶水脚本则被削弱⁴

2.2 被削弱的”胶水”与被强化的”底层”

坤哥的核心论点可以归纳为一个二分法：AI 削弱的是**胶水型脚本工具**，强化的是**底层能力工具**。

所谓胶水型脚本，指的是那些以”把流程拼起来”为主要价值的工具，例如简单发包、正则匹配、字段提取、把若干 CLI 命令按固定顺序串起来等。这一类工具之所以会被削弱，是因为它们的价值密度较低，大模型在零样本条件下就能直接生成等价的 Python 或 shell 代码，用户无需再保留它们作为独立资产。换言之，AI 是在与胶水脚本竞争同一个生态位，竞争的结果几乎注定。

而底层能力工具则不同，它们封装了模型本身无法在生成时凭空补出的硬能力。例如坤哥团队在客户反馈中频繁听到的一个例子：用户用 Yakit 的产品可以发国密（国产密码学算法）的数据包，可以扫描部署了国密协议的网站，但用普通的 KO（如普通的 Burp 或网上下载的开源扫描器）就完全做不到——这不是因为脚本写得差，而是底层根本没有实现国密协议栈。在这种场景下，把任务交给纯大模型同样无法解决：因为密码学的生成、加解密、验证情绪安全等操作具有严格的数学正确性要求，任何一个 bit 的错误都会导致请求失败，模型再聪明也不能”幻想”出一个正确的密文。这一类底层能力的不可替代性，恰恰因为 AI 的介入而变得更加凸显——**模型有多强，依赖的下层工具就要多稳**。

⁴视频画面时间区间：00:14:45-00:14:55。

值得注意的是 AI 自身的工作机制其实也在印证这一论点：从 API 层面看，AI 大部分时间都在做”问答”或”问答 + ToolCall”，所谓 ToolCall 也只是返回一段 JSON 描述要调哪个函数、传什么参数，真正的发包、加解密、运行环境调用，仍然落到底层工具上。模型像是一个聪明但”手不能动”的指挥者，它的指令最终必须由可靠的执行器来兑现。

胶水型脚本的退场

”用正则解析输出”、”把若干命令拼成一行 bash”、”硬编码请求模板”——这一类胶水型脚本会被 AI 直接生成或替代，独立维护它们的工程价值正在持续走低。对工具开发者而言，更稳健的方向是转向那些不可被模型零样本复刻的底层能力：协议栈、密码学、内核交互、确定性沙箱、长尾硬件接口。**越是依赖具体物理或数学正确性的能力，越不会被模型替代。**

2.3 工具的新角色：能力接口与执行槽

赵一凡接着从 Agent 设计者的视角对坤哥的论点做了延伸。在他主导的 CyberStrike AI 中，工具不再是人类亲手操作的对象，而是被大模型组织进决策链路里的一个个执行槽。”过去工具更像是一个能力的本体，本身是人去操作的对象，比如人去操作加密算法的工具；但是在 AI 介入之后，工具更像一个能力的接口和执行槽。”换句话说，工具的形态没有变，但它在系统中扮演的角色已经从”被使用”转变为”被调度”。

这种转变的意义不容低估。”被使用”意味着工具的接口、参数、错误处理都为人类设计，对模糊提示和容错率有较高的容忍度——人脑可以即时修正一次错误的命令；而”被调度”意味着工具需要面对一个对参数语义高度敏感、却又会偶尔产生幻觉的调度者，错误的输入既不能崩溃，也不能默默吞掉，必须以机器可解析的方式向上回馈。这是一种全新的接口契约。

更重要的是，工具不再是唯一的执行主体。以前的渗透流程中，所有决策与执行都通过人这一条线收口；而在 AI 时代，AI 与人共同构成调度者，工具变成了双调度链中的共享资产。Agent 框架（例如 CyberStrike AI）会按上下文自动选择工具、组织顺序、解释返回；人类则在边界上做监督、注入约束或接管特殊场景。这种”双驾驶舱”格局带来的直接结果，是工具开发者必须同时为两类用户写文档、做接口、做错误码——给人看的与给 AI 看的标准不再天然重合。

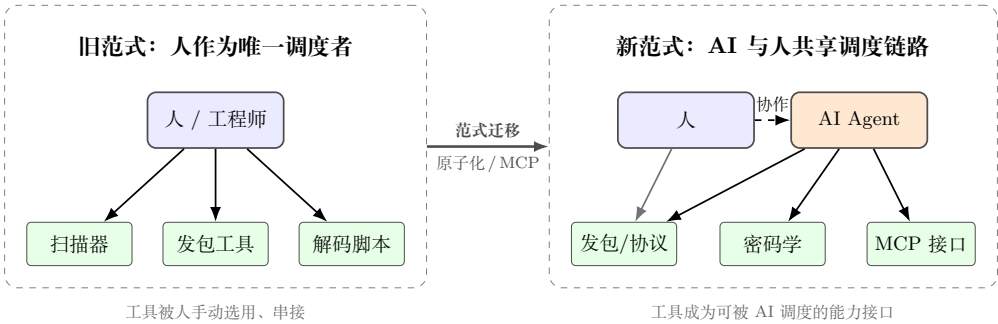


图 5: 工具角色迁移示意：人作为唯一调度者 → AI 与人共享调度链路⁵

⁵ 本图为概念示意图，非视频原画。

工具的新定义

在智能渗透范式下，工具 = 提供给 AI 的确定性能力接口。它的设计目标不再是”让人用得顺手”，而是”让 AI 能稳定调度”。这意味着：

- **原子化粒度**：能力被拆成可独立调用、可自由组合的最小单元，而非固定大流程；
- **稳定性优先**：错误码、超时、重试策略要对模型可读、对调度器友好；
- **语义清晰**：参数名、返回字段必须在语义上无歧义，避免模型误解；
- **可组合性**：工具间的输入输出能够顺畅地形成 pipeline，不需要外部胶水。

2.4 平台视角的佐证：从单一 API 到 MCP，再到原子化能力

如果说坤哥强调的是”工具底层能力被抬高”、赵一凡强调的是”工具角色被调度化”，那么作为比赛平台开发负责人，齐涵从**基础设施**的角度提供了第三层佐证。

齐涵观察到的最大变化是：以前的工具基本上”只为人服务”或”只为固定程序服务”，而现在的工具必须显式考虑被 AI 使用这一新身份。这一变化对工具的质量、稳定性与精准性提出了远比以往严苛的要求。在第一届智能渗透挑战赛中，靶场只提供了一种 HTTP API 形式的接入方式；到了本届，平台主动新增了 MCP 形式的接入接口，让参赛 Agent 能够通过更标准化的协议层接入靶场，无需逐题手写 API 适配代码。这一调整背后的判断很直接——**基础设施需要向 AI Agent 友好的方向倾斜**，否则就会成为参赛者的瓶颈。

更进一步，齐涵团队在自己日常的攻防工坊工具中也进行了类似改造。过去的攻防基础设施往往按照”资产测绘 → 外网打点 → 漏洞探测 → 后渗透”这条固定流水线组织能力，本质上是一条线性 pipeline。而在 AI 时代，这种死流程会成为枷锁——因为 AI 真正擅长的，是面对开放空间动态决策应该走哪一步。于是新一代基础设施开始按**原子化能力 (Atomic Capability)**的思路重新组织：把资产测绘、漏洞探测等大流程拆分为数十甚至上百个可独立调用的最小单元，让 AI 像搭积木一样根据当前任务上下文自由重组，而不是被动地走完固定工序。



图 6: 比赛平台从单一 API 升级到 MCP，能力被拆分为可重组的原子⁶

⁶视频画面时间区间：00:20:40-00:20:50。

这种转变并非毫无代价。原子化拆分意味着每一个细胞级能力都要独立保证质量、独立暴露语义化接口、独立维护文档与错误码——开发与维护成本不降反升。但这是被 AI 时代的工程压力倒逼出来的结构性进化：当工具从”为人设计”切换到”为 AI 调度设计”时，质量门槛自然水涨船高。换言之，看似工具的”用法”变了，但深层是工具的**设计目标人群**变了，而设计目标人群一变，整套度量、流程、组织方式都要随之调整。

2.5 本章小结

本章回应了一个普遍存在的直觉误区——AI 时代工具会被边缘化。从坤哥的反例（普通工具发不了国密包、AI 也救不了底层能力的缺失）出发，我们看到 AI 削弱的只是胶水型脚本，而抬高了底层能力工具的不可替代性；从赵一凡的论断出发，我们理解到工具的角色从”被人使用”迁移到了”被 AI 调度”，工具变成了能力接口与执行槽，调度权由人和 AI 共享；从齐涵的平台实践出发，我们看到比赛靶场从单一 API 升级到 MCP，攻防基础设施从固定流水线拆分为原子化能力，工具的稳定性与精准性要求被 AI 介入显著抬高。三种视角共同指向同一个结论：**工具不死，工具被重新定义**——它从人手中的趁手家伙，进化为 AI 调度链路上承担确定性执行的关键节点。理解了这一点，我们才有可能在下一章中讨论清楚：当模型与工具同时存在时，比赛胜负的真正决定者究竟是谁。

3 模型 vs 工程化：上限由模型决定，下限由 Scaffold 决定

上一章我们看到工具的角色被重新定义为 AI 调度链中的执行槽与确定性接口。一个自然的反问随之而来：既然工具与 AI 同时参与调度，那么决定一个 Agent 战力的究竟是模型本身、还是模型外面那一圈**工程化能力 (Engineering Capability)**？嘉宾们在这二十多分钟里给出的答案颇为反直觉——决定胜负的并不是『谁拿到的模型更大』，而是『谁把模型外面的工程脚手架搭得更稳』。本章按照『趋势观察 → 自动化 vs 智能化 → 模型差异化 → 同模型两极现象 → 工程化为王 → 外挂式 Agent 框架 → 分工总结 → 幻觉治理』的顺序，把这条反直觉链条逐节拆开讲透。

3.1 趋势观察：从”AI 生成文本”到”AI 完成任务”

坤哥在被问及工具调度时，先抛出了一个被很多人忽视的事实：所谓 AI，其本质仍是文本生成；它调工具的动作，归根结底也只是生成一段 JSON。真正发生变化的，是围绕这段文本生成所建立的工程范式：从 ReAct 的”思考-行动-观察”循环，到 Plan-and-Execute 的先规划再执行，再到最近开始流行的、带有控制论色彩的”假设-探索”——先假设一个前提，然后在可能性空间里一步一步搜索，每一步都借助外部工具或外部知识来收敛。



图 7: 三种主流 Agent 调度范式：ReAct、Plan-and-Execute、假设-探索⁷

这个变化的更深含义在于”默认值”的迁移：去年人们对大模型的默认理解还是”它能生成文本”，今年的默认理解已经变成”它可以做事情”。一旦默认值变成”做事”，整个讨论焦点就自然转向 Agent 架构、上下文如何维护、何时压缩、何时追加知识——这些都是工程问题，而不是模型问题。

3.2 自动化 vs 智能化：方法论的分水岭

李鑫顺着这个话头进一步把”自动化”与”智能化”做了切分。**自动化**是基于规则的、确定性的流程化执行：传统扫描器把所有规则跑一遍，匹配命中后给出漏洞标签。它的优势是稳定可重复，但缺陷也十分明显——规则会越堆越大，路径或参数稍有变动就匹配不到。**智能化**则依赖模型本身的感知能力，通过抓取页面信息、端口信息、版本信息等线索做语义层的推测，再去尝试可能的漏洞。

⁷ 视频画面时间区间：00:23:05–00:23:15。

由此，比赛中出现了两种典型解题路径：一种是把 AI 当成“扫描器的使用者”，让 AI 去操作传统工具；另一种是让 AI 直接基于感知去推测、再去验证。这两种路径的胜负差异，恰恰预示了后文将要讨论的“工程化”主题——自动化决定能否复用，智能化决定能否泛化。

3.3 模型差异化：训练语料决定的风格分野

在进入“工程化为王”的核心论点之前，嘉宾们也承认：**模型本身的差异是客观存在的**。学员反馈中“国外模型成绩更好”的现象并不是错觉。坤哥指出，国外模型在训练语料里给计算机相关内容分配的比重明显更高，而国内模型出于产品定位放入了更多文学、人文类语料，于是在长链路、强逻辑的攻防任务里二者风格出现差别。

这个差距说明：**模型是上限**。在解题这一封闭任务上，模型决定了“理论上最多能跑多高”。但模型并不是唯一变量，下面要看的“同模型两极现象”才是真正撕开工程化重要性的那道口子。

3.4 同一个模型，为什么会口碑两极？

主持人段刚抛出的关键问题是：同一个 Kimi 模型，为什么有人说“完全不够用”，有人却说“Kimi 2.5 已经足够强”，而且本届比赛排在前面的不少队伍恰恰用的就是 Kimi。

这里先要直面一种错误直觉：如果有人觉得难用，那一定是模型不够好。坤哥的反驳很直接——同样一把刀，不同的人切出来的形状会完全不同。差异从来不在刀身，而在握刀的手；这双手，就是 Agent 的工程实现。一支队伍可能只用了一个手搓的 for 循环式的小型 ReAct，没有记忆、没有上下文压缩、没有工具兜底；另一支队伍则配了完整的记忆模块、动态工具推荐、verification 校验。同样的 Kimi 在两种工程化水平下跑出的成绩自然天差地别。

由此引出本章的核心定律：

上限 vs 下限定律

模型决定 Agent 的**上限**：再精妙的工程化，也无法让一个能力较弱的模型在长链路推理任务中突破其训练所赋予的天花板。

工程化与**工具兜底 (Tool Fallback)** 决定 Agent 的**下限**：再强的模型，如果交给一个没有记忆、没有兜底、没有 verification 的劣质 Scaffold，也会在分数衰减机制下被慢慢拉开差距。
真实战力 = 上限 × 下限——选模型决定天花板，搭 Scaffold 决定地板。

3.5 工程化为王：5-payload 的算账例子

抽象的“上限/下限”听起来仍然有点空，坤哥给了一个非常具体的算账例子，让“工程化为王”这件事在数字层面变得可触摸。

设想一个漏洞需要从 **5 个 payload** 中挑出唯一能打通的那一个：

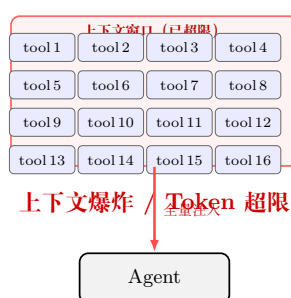
- **纯 AI 路径**：模型根据特征观察后挑一个去打。第一次命中的概率为 $1/5 = 20\%$ 。即便是更强的模型，命中概率提升到 50% 甚至 70%，跑十次都过的现象也只是“概率够高”，并不等于“百分百锁定”。更糟的是，AI 还要保证十几个相关请求都被正确串联，全程可能要花 30 秒甚至 1 分钟。
- **工具兜底路径**：如果基础工具的批量发包模块本身就内置了这条 payload，那么“对的那一个”在工具一次扫描中就能被命中，三秒级出结果。剩余的 4 个 payload 是否需要变形、要不要新增组合，再交给 AI 在 verification 阶段补刀。

把它写成一笔粗略的“分数账”：假设比赛满分 100 分，工具兜底直接锁定 50 分的确定性子集，AI 在这 50 分之上再做不确定性增益（比如再覆盖 30–40 分的变体路径）。这就是确定性 + 不确定性的乘法效应：

$$\text{得分} = \underbrace{\text{工具兜底基线}}_{50 \text{ 分稳定下限}} + \underbrace{\text{AI verification 与扩展}}_{0-50 \text{ 分浮动上限}}$$

而纯 AI 路径意味着把整个 0–100 分的可能性空间都丢给模型去赌——上限可能很高，下限却完全暴露在幻觉与算力波动之下。坤哥在现场还顺手举了一个补充观察：最近大家普遍感觉某些大模型的“算力被降了”，同一段编码任务的表现明显不如以前——这说明纯 AI 方案的下限其实并不掌握在使用者手里。

左：把全部工具一次性塞进上下文



右：意图感知按 30 秒粒度动态推荐

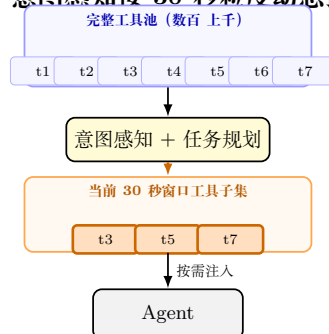


图 8: 工具池规模膨胀时，意图感知按 30 秒窗口动态注入工具，避免上下文爆炸⁸

与上述算账紧密相关的是意图感知与任务规划 (Intent Perception & Task Planning)：当工具只有 10 个时，可以把全部工具描述塞进 system prompt；但工具一旦扩展到 100 个、1000 个，预设全部就会瞬间打爆上下文窗口。坤哥介绍 Manfate 即将更新的核心思路便是——根据当前任务意图，预测“接下来 30 秒内”应该向 AI 推荐哪些工具子集，让模型每一步只看到自己真正用得上的小池子。这是把超大工具池变得可调度的工程前提。

同样值得对比的是 Cloud Code 与 Cursor：底层是同一类前沿模型，开发者却普遍感觉两者“是两种感觉”。差别不在模型，而在工程基础——上下文如何组织、工具如何调度、记忆如何维护、错误如何回退。这进一步印证了：工程化才是同模型下真正的变量。

3.6 外挂式 Agent 框架：放大模型上限的关键设计

讨论到这里，李鑫引入了一篇南开大学方向的论文作为理论侧的佐证。该论文的核心观点是：Agent 的能力不应该取决于“换一个更大的模型”，而应该依赖一种『外挂式 Agent 框架 (Scaffold-based Agent)』的设计取向——把记忆、Cursor 类工具集、可控沙箱、MCP 协议、verification 校验等模块以外挂形式补到模型外部，让外挂去把现有模型的能力充分发挥出来。

这条结论同样反直觉。一般人的第一反应是：模型不够强？那就等下一代更大的模型。然而论文与嘉宾共同强调，更现实、更可控的路径是把模型外面的脚手架补完：

⁸本图为概念示意图，非视频原画。

外挂式 Agent 框架 (Scaffold-based Agent) 的关键组件

- **记忆模块**：把比赛过程中收集到的碎片化情报、凭据、路径信息分类存储，供后续步骤随时检索；这是嘉宾反复强调”比赛中最关键能力之一”。
- **Cursor 类工具集**：在模型知识库缺失之处补上检索式知识增强，弥补国产模型在某些专业语料上的不足。
- **可控沙箱**：为模型提供可信、可控、可审计的执行环境，避免在真实系统上直接试错。
- **MCP 协议**：当模型缺少某类工具时，用统一接口接入已有工具生态，让”缺工具”成为可填补的工程问题。
- **Verification 机制**：对工具与模型的中间结果做二次校验，决定是否需要补打、补扫、补判断。

外挂式 Agent 框架的精神可以一句话概括：**不靠换模型来追上限，而是靠搭脚手架来抬下限**。一旦下限被抬到足够高，模型与模型之间的能力差异，就会被工程化能力反向放大。



图 9: 外挂式 Agent 框架：记忆、Cursor、沙箱、MCP 共同放大模型上限⁹

3.7 分工总结：确定性归工具，不确定性归 AI

赵一凡把整章的方法论凝结成了一对极其干净的对偶：

- **工具 = 执行空间，承担确定性能力 (Deterministic Capability)**。发包、指纹识别、加解密、POC 投递这类可重复、可验证、结果稳定的动作，应交给工具。
- **AI = 搜索空间 + 决策空间，承担不确定性推理 (Uncertain Reasoning)**。哪条路径更高效、哪个异常需要回溯、哪条线索值得二次探索，是 AI 推导推理推演的舞台。

齐涵在这一节做了重要的范式纠正：常见说法”能用脚本解决就别用 AI”其实把工具与 AI 摆在了同一个 level 上，但真正合理的关系是”AI 是工具的使用者”——模型像一位指挥者，借助工具

⁹视频画面时间区间：00:33:45–00:33:55。

去发请求、做指纹识别、投递 POC，而它自身则负责站在决策层判断路径优劣、回溯异常、规划下一步探索。这与赵一凡的对偶完全互补，把”AI 与工具谁先谁后”的争论转换为”调度者 vs 被调度者”的层级关系。

这种分工的工程含义在于：**把整个可能性空间分成两个互补的子空间**。确定性子空间被工具”锁定”，AI 不必为之耗费 token 与时间；不确定性子空间被 AI”探索”，工具不必为之硬编码穷举规则。两者通过 verification 相互校验、相互兜底。

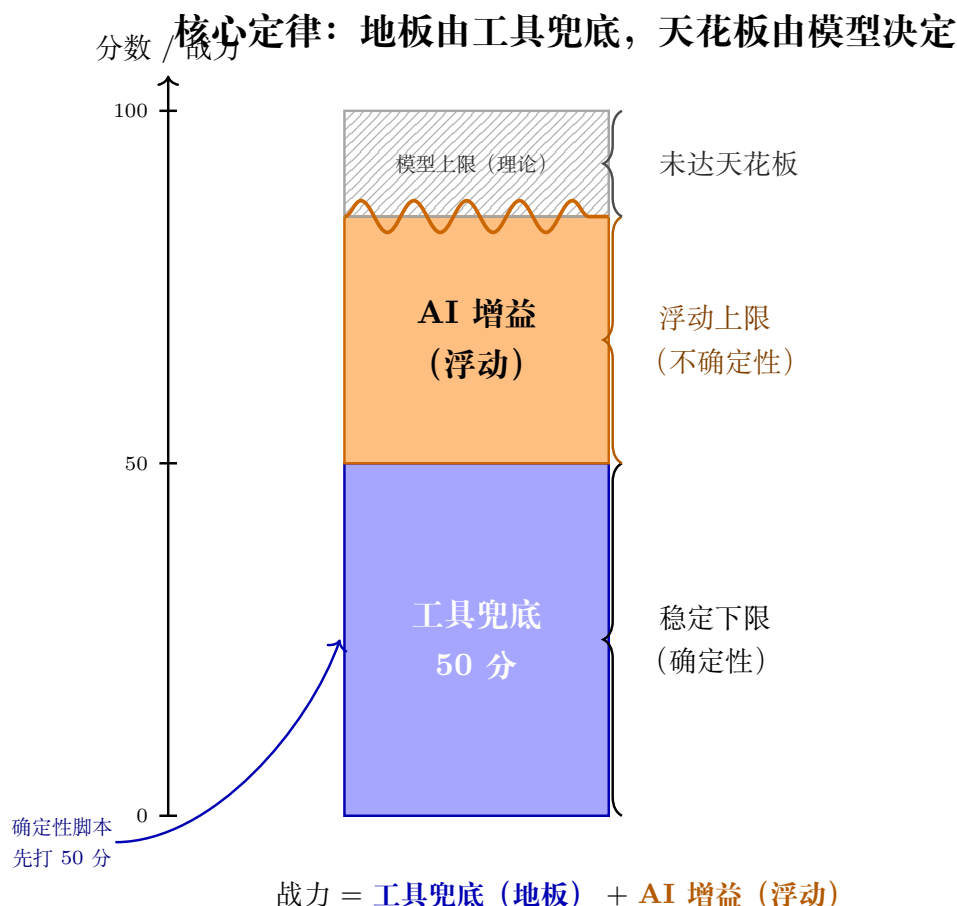


图 10: 工具兜底 50 分 + AI 增益 = 稳定下限 + 浮动上限¹⁰

3.8 幻觉治理：工具兜底与二次 verification

主持人接着抛出最后一个具体问题：AI 存在不稳定性，也就是大家常说的幻觉——这次能解出来的题，下次可能就解不出来。这究竟是模型能力的问题，还是工程设计的问题？

齐涵的现场观察十分有意思：模型在某些题目上”一请求就有思路”，把工具串联起来一气呵成；但一旦遇到没有现成解题流程的题，或者赛题里精心埋下的”看似可走、其实走不通的陷阱路径”，它就可能反复尝试、陷入死循环——这就是工程意义上的幻觉表现。

嘉宾给出的方案不是”等待更聪明的模型”，而是从工程上把幻觉”包起来”：

¹⁰ 本图为概念示意图，非视频原画。

幻觉治理的工程手段

1. **先工具粗扫，确定性结果先到手。**让基础扫描器、批量发包工具、规则引擎先把能拿的分稳稳拿到，避免把整张试卷交给概率。
2. **再让 AI 做 verification。**把工具的结果回投给模型——发了几个包、命中哪些、漏了哪些、是否存在赛题陷阱——由 AI 在小范围内做语义校验。
3. **有针对性地补打不确定空间。**在 verification 之后，让 AI 自己写新的 payload 去补漏、绕过陷阱路径，模型聚焦在”剩余未解空间”上，命中率显著提高。
4. **绝不二元选择。**不是”用了工具就不让 AI 决策”，也不是”用了 AI 就抛弃工具”；二者必须叠加，把确定性与不确定性的乘法效应吃满。
5. **警惕算力波动。**模型供应商侧的资源波动会让纯 AI 路径”今天行、明天不行”。工具兜底是抵抗这种波动的最实用手段。

需要强调的是：**幻觉不是要被”消灭”的，而是要被”圈住”的。**把幻觉关进工具兜底与 verification 这两道围栏里，下限就被稳稳锁住；剩下的不确定性，反过来正是 AI 真正擅长的事情。

3.9 本章小结

把这一章的反直觉链条再串一遍：

- AI 的本质仍是文本生成，调度范式（ReAct、Plan-and-Execute、假设-探索）决定了它能多高效地”做事”。
- 自动化追求基于规则的稳定，智能化追求基于感知的泛化，二者不是替代而是分层叠加。
- 模型本身存在客观差异——训练语料决定了风格分野——这决定了 Agent 的上限。
- 但同一模型在不同团队手中口碑两极的事实证明：决定胜负的真正变量是工程化能力，而不是模型本身。
- 5-payload 的算账例子从 1/5 概率出发，演示了工具兜底如何把”概率赌博”变成”确定性 + 不确定性”的乘法效应。
- 南开大学的论文观点为这一现象提供了理论侧的支撑：Agent 上限靠外挂式 Agent 框架（记忆、Cursor、沙箱、MCP、verification）放大，而非靠换更大的模型。
- 工具与 AI 的合理分工是确定性归工具、不确定性归 AI；AI 是工具的使用者，工具是 AI 的执行槽。
- 幻觉问题不是要消灭，而是要圈住——先工具粗扫、再 AI verification、再针对性补打，是当下最稳的工程治理范式。

把这八条凝成一句话：**选模型决定你能跑多高，搭 Scaffold 决定你能站多稳；只有两者乘起来，才是真正的 Agent 战力。**下一章将沿着这条主线，从”封闭比赛”走入”真实攻防”，看看在更碎片化的战场上，这套方法论还会遭遇什么样的挑战。

4 能力边界：从 CTF 解题到真实攻防，AI 还差最后一公里

前一章我们讨论了上限与下限，结论是：模型决定上限，工程化与工具兜底决定下限。沿着这条线再往前推一步，自然会问到一个更现实的问题——当 Agent 在比赛靶场上把分数刷得越来越高时，它到底离一个能在生产环境中独立完成『一条龙』渗透链的工程师还差多远？这一节我们把这段距离称作智能渗透的『最后一公里』，并从环境、任务粒度、协作模式三个层面把这段差距讲清楚。

4.1 比赛环境与真实攻防的结构性差异

参赛靶场是一个『相对独立』的世界。它会把目标、入口、可达资产、可用接口都梳理得相对干净，再设计成一条由步骤一到步骤十依次推进、最终拿到 flag 的线性流程。规范流程换来的是公平评判，这是赛事必须坚持的设计取舍。但真实工作并不长这样。

坤哥在现场举了几个让人立刻有共鸣的画面：团队三个人合作做一个项目，你只负责打点，需要小心谨慎地把数据收集、整理好交给下一棒同事；很多用户在产品群里抛过来的『渗透测试』，最后只是一个数据包——可能来自老板、组长，也可能直接来自甲方——要求你把这一个接口测完并给出结论；甚至还存在一类完全不允许发包、只给二十多 GB 历史流量的蓝队/红军响应任务，需要你在天眼离线包里定位潜在问题。这些任务都有一个共同特征：碎片化。



图 11: 真实工作场景的碎片化：只给一个数据包、一段流量、一个国密接口¹¹

碎片化意味着没有一条预先铺好的剧本，也意味着 CTF 式的『从头跑到尾』思维并不天然适配。对照之下我们可以这样总结两者的差异：CTF 是一条链路、十个连续步骤组成的规范沙盒；真实攻防则是一堆散点、谁递给你什么你就只能就着什么干的非结构化战场。

¹¹ 视频画面时间区间：00:45:55-00:46:05。

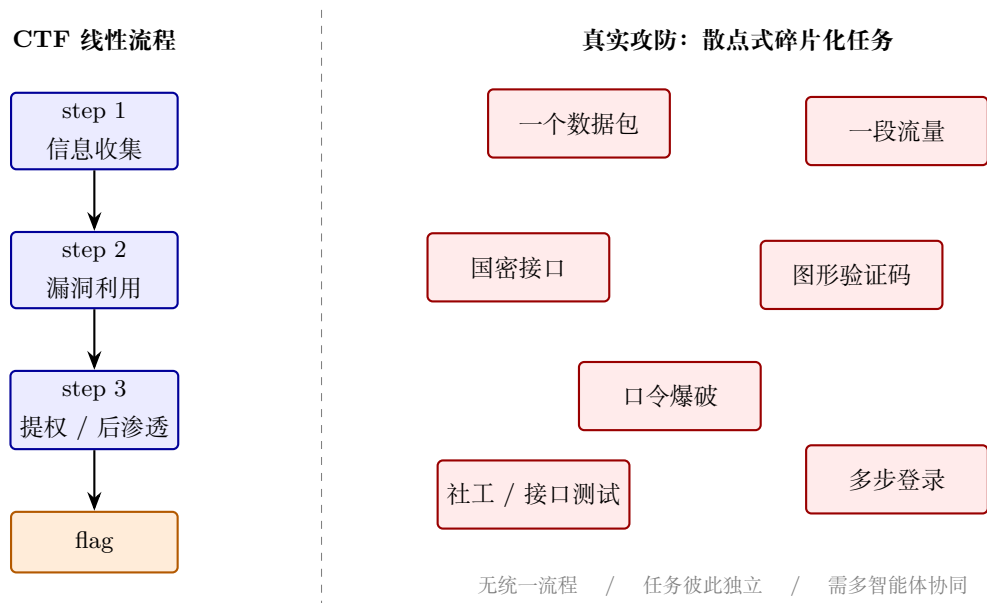


图 12: CTF 线性流程 vs 真实攻防的散点式碎片化任务¹²

4.2 杀鸡用牛刀：当大 Agent 反向拉低效率

碎片化场景天然不适合『召唤一只大型 Agent』来全自动接管，原因可以归纳成三点。第一，启动成本不对称。一个有完整 Scaffold 的智能体在跑起来之前要拉基础设施、加载工具池、构建上下文，本质上是个『重型流程』。第二，任务尺度不匹配。当真实任务只是改一个 payload、跑一次国密接口时，所谓推理链反而成了冗余。第三，确定性溢出。能用一个发包脚本几秒钟解决的事情，丢给模型反而引入了模型本身的不确定性与算力波动。

坤哥在现场给了一个非常具体的反直觉例子：一件本来 5 秒钟就能搞定的小事，套上 AI 之后耗时变成 1 分钟左右。简单换算一下，效率下降大约 20 倍。这就是『杀鸡用牛刀』的工程含义——不是 AI 不行，而是任务太小。再叠加碎片化任务里常见的国密协议（普通 Python/Go 脚本写起来就别扭，让 AI 去补依赖更慢），效率倒挂会进一步放大。

杀鸡用牛刀的反直觉

任务尺度小于 Agent 启动成本时，套用大 Agent 反而拉低效率。一个可操作的判断规则是：

- 任务可以用现成发包工具（Yakit、Burp Suite）改 payload 在数十秒内完成 ⇒ 直接人手或脚本，不上 Agent。
- 国密、私有协议等小尺度但带专业能力门槛的任务 ⇒ 优先选择已有底层能力工具，必要时把数据包直接喂给 AI 让其改写脚本，避免拉起完整 Agent。
- 长链路、跨工具、跨子领域协作的任务 ⇒ 才是大 Agent 能放大产出的真正主场。

口诀：5 秒能解决的事，不要用 1 分钟的 Agent 解决。

¹²本图为概念示意图，非视频原画。

4.3 第三赛区第三题：被『真实场景』难住的团队

为了让靶场更靠近真实攻防，平台方在设计本届题目时特意把一些『过去靠人工经验渗透下来的活儿』搬了进来。最典型的的就是第三赛区第三题：包含图形验证码爆破、登录后的多步操作等真实业务里非常常见的环节。结果是大多数团队在这一题上卡住，并不是题目本身有多难，而是这些细节恰好踩在了当前 Agent 不擅长的领域。



图 13: 模拟真实场景的赛题：图形验证码爆破 + 多步登录操作¹³

把这道题放大看，会发现人类工程师在真实攻防中的几个 AI 暂时难以替代的优势：第一，执着抠细节。人会盯着一个看似走不通的小入口反复试、反复换思路；AI 走一条路走不通就倾向于直接判定『此路不通』。第二，互动与灵感。人类团队之间会随手抛出一句『这个东西怎么弄』，灵感往往来自这种非结构化对话；AI 在没有明确触发条件时不会主动跳出当前推理链。第三，跨子领域调用。需要某个网站的密码时，人类可以直接走社工路径——不是 AI 不能社工，而是把社工塞进每一次口令爆破环节里，对单一智能体来说成本极高，整条链路会被严重拖慢。

4.4 多智能体协同：CTF 升级到真实攻防的过渡形态

把上述短板拼起来，就能自然推出下一阶段的演进方向：与其逼一个大模型在同一上下文里又社工又爆破又识别验证码，不如把每一类子任务交给专门的智能体，再由调度层完成协作。这就是**多智能体协同（Multi-Agent Collaboration）**在安全语境下的具体含义。

¹³视频画面时间区间：00:49:55–00:50:05。

多智能体协同的下一站

社工 Agent: 维护社工话术库与目标人群画像，专门处理钓鱼、口令猜测、身份伪装等需要语义与人际推理的子任务。

口令爆破 Agent: 管理字典、速率控制、账号锁定策略，对接底层发包工具完成确定性执行。

验证码爆破 Agent: 调度图形验证码识别模型、滑块/点选识别能力，并与登录流程的状态机联动。

调度层: 根据当前主任务的语义把子任务分派给上述 Agent，并维护共享记忆与回退路径。这种设计把『一个大 Agent 什么都干』变成『一组小 Agent 各自精专』，对应到本讲义的术语体系，就是把原子化能力进一步细化到智能体粒度，让真实攻防中的散点任务能被分别承接。

4.5 靶场设计的两难与下一届 Benchmark 承诺

把真实场景搬进比赛并不是免费午餐。李鑫提到，平台方在设计靶场时其实犯过难：那些过去靠人工经验趟下来的『脏活儿』敢不敢放进来？放进来之后还能不能保证可量化、可对比的评判？这是一对结构性矛盾——越逼近真实，规范度就越难保住；越追求公平，越容易回到 CTF 式的线性沙盒。本届的解法是有限度地引入真实环节（如第三赛区第三题），观察 Agent 卡在哪儿，由此反推能力边界。

更进一步，平台方明确把『考察 Agent 的能力边界』列为下一届比赛的设计核心：将多智能体协同、社工类任务、流量分析等真实场景中的核心问题写入靶场考察项，构建一个能更完备地度量安全 Agent 真实战力的 Benchmark。这一步本身就是把行业从『拼解题速度』推进到『拼真实可用度』的关键。

4.6 本章小结

本章的核心论断只有一句：会做 CTF 不等于会做真实攻防。原因是真实工作具备碎片化、协作化、跨子领域的结构特征——只给一个数据包、一段流量、一个国密接口的场景非常常见，而对这种小尺度任务盲目套用大 Agent 会造成 5 秒变 1 分钟、效率下降约 20 倍的反向损失。第三赛区第三题（图形验证码爆破叠加多步登录）正是把『人类工程师抠细节、互动、社工』的优势点暴露出来的真实例子。下一阶段的演进方向是把社工、口令爆破、验证码识别等子任务拆给各自的专门智能体，由调度层完成多智能体协同；下一届比赛也将把这些真实场景化的考察项纳入 Benchmark，让能力边界的度量更完备。读者从本章应该带走的最核心判断依据是：选不选 AI，先看任务尺度——任务越小越确定，越应该让工具直接上；任务越长越开放，多智能体协同才是它真正的主场。

5 开源的双刃剑：站在阳光下做工具

讨论完能力边界，话题自然滑到另一个绕不开的命题：开源。本届圆桌的两位作者分别带来了 Yakit（万界安全的开源安全平台）和 CyberStrike AI（个人开源的安全 Agent 项目），它们在推动行业技术进步的同时，也无可避免地降低了一部分攻击门槛——这就是**双刃剑效应（Double-edged Sword）**。本章要回答的问题不是『要不要开源』，而是『开源的人怎么把自己放在阳光下』，以及当舆论先于辩解到来时，作者还能不能自证清白。

5.1 Yakit 早期的合规焦虑：恐惧从何而来

时间倒回两三年前。坤哥回忆起 Yakit 团队在那段时间里反复被一个问题困扰：万一有黑产用我们的工具去打了政府或关键基础设施的网站，公司会不会被牵连？我们会不会被认定为『军师』？这种焦虑甚至蔓延到了员工内部——做工具的人比用工具的人更怕。这种焦虑是真实存在的，今天每一位准备把攻防类工具开源的开发者都会面对类似的拷问。



图 14: 坤哥：站到阳光下、用用户量与资质换取信任¹⁴

那为什么这种焦虑后来淡了？坤哥转述了一位前辈给他讲过的观点：你要站在阳面，要站在太阳光下，把自己的东西交给别人去看，去换取别人的信任。这句话点破了开源安全工具的合规哲学——不是把刀藏起来就安全，而是把刀放在超市货架上，让所有人都看到它怎么被合规地使用。

5.2 阳谋四件套：把工具放在阳光下的具体操作

把『站在阳光下』翻译成可执行的开发者动作，可以归纳成四件套：用户画像、场景定义、合规资质、防守视角传播。这四件套并不是修辞，而是相互支撑的一组合规机制。

¹⁴ 视频画面时间区间：00:55:15-00:55:25。

阳谋四件套

1. **用户画像**。明确这个工具是给谁用的：是企业蓝队、内部安全评估团队，还是授权红队？把目标用户群写进 README、产品页与对外稿件，让外界一眼看出工具的合理使用人群。
2. **场景定义**。把『使用场景』写死在工具介绍里：定位为安全评估工具、企业风险测评工具，而不是泛泛地说『渗透神器』。场景越清晰，被恶意利用时作者的辩护空间越大。
3. **合规资质**。用户量、专利、高新企业认证、三所证书、关键部门合作等共同构成合规护城河。当国企员工、银行员工、大型科技公司资深用户都在用这个工具时，工具就走上了正轨——它在制度上有了背书。
4. **防守视角传播**。公众号、文章、案例统一切换到企业安全建设与防御视角，强调『如何帮助企业评估网络』，而不是反复展示攻击成功的画面。

坤哥用一个生活化的比喻把这层逻辑挑明：菜刀。绝大部分人买菜刀回去做饭，少数人拿菜刀做坏事——出了事情，那是个人的责任，不是菜刀厂商的责任。但要让这个类比真正成立，前提是这把刀必须摆在公开超市里、有正规渠道、有清晰用途说明，而不是藏在地下交易市场。把这层前提补齐，开源安全工具的合规叙事才完整。

开源安全工具的四步合规闭环

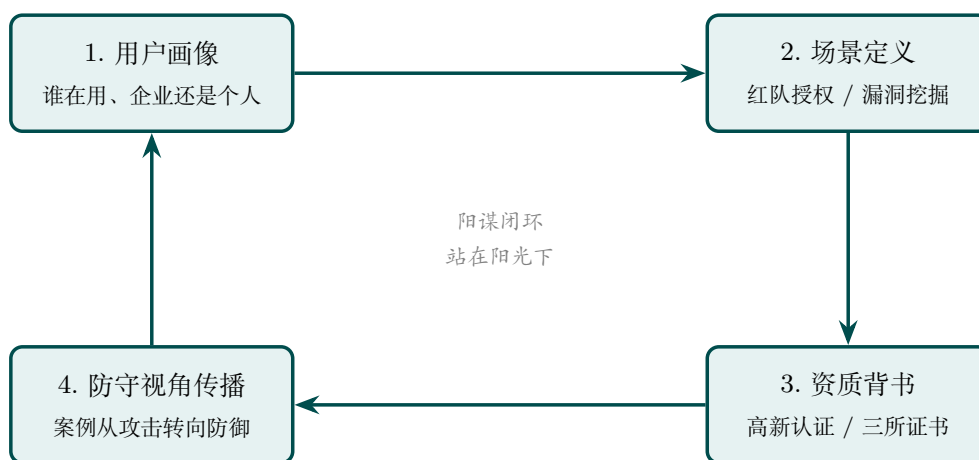


图 15: 开源安全工具的四步合规闭环¹⁵

5.3 舆论先于辩解：海外报道带来的真实威胁

阳谋四件套之所以必须前置而不是事后补救，是因为舆论一旦起来就再也压不下去。今年发生过这样一件事：海外媒体报道某团队使用 CyberStrike AI 扫描飞塔（Fortinet）漏洞，拿下了全球数以万计的资产；国内自媒体随后跟进起哄，视频号上的二次传播甚至带上了相当惊悚的标题。事件本身的真实性如何另当别论，关键是它揭示了一种结构性威胁——只要海外把『国内开源工具被 APT 使用』的话术抛出来，国内开源作者就被动地站到了被审视的位置上。

¹⁵ 本图为概念示意图，非视频原画。

舆论先于辩解

工具被媒体定性的速度，永远快过作者澄清的速度。一旦舆论起来，作者再去解释技术细节、还原事实经过，往往已经晚了。这意味着合规与传播的防御必须前置：

- 在工具尚未被点名之前，就把用户画像、使用场景、免责条款、授权范围全部写到位。
- 公众号与对外稿件要长期沉淀『防守视角』的素材，构成可被引用的反证语料。
- 与关键部门、企业用户的合作要有可公开的台账证据，关键时刻能够拿出来背书。

不要相信『等真出事了再解释』，那时候你已经没有舞台。

5.4 个人开源作者的最低自保：免责声明 + 授权范围 + 禁用滥用

Yakit 是公司产品，可以靠资质堆出合规护城河；CyberStrike AI 这种『个人开源软件』情况略有不同——没有公司资质背书，作者必须靠声明与边界把责任划清。赵一凡给出的做法可以概括为三件套：

1. **免责声明**。在仓库与文档显著位置写明，作者不对工具被滥用造成的后果负责。
2. **授权范围**。明确声明工具仅供授权红队渗透测试与授权漏洞挖掘使用。把『授权』两个字写到台前，是把使用者推向责任主体的关键动作。
3. **禁用滥用**。显式禁止通过本工具从事任何未授权或非法行为，并在 README 与发布说明中重复出现。



图 16: CyberStrike AI 通过免责声明与授权范围维持合规边界¹⁶

赵一凡补充了一个更上层的视角：开源确实降低了一部分能力门槛，但本质上不是让世界变得更危险，而是攻防双方的同起点被同时抬高了。换句话说，一个工具公开后，攻击者拿到了更便宜的武器，防御者也拿到了更便宜的检测样本与防御参考。从行业基线的视角看，这是一次共振式的提升。

¹⁶ 视频画面时间区间：00:57:50-00:58:00。

5.5 公众号写法的常见误区：只写攻击就是把出发点暴露

李鑫还点破了一个开发者最容易踩的雷：很多白帽子作者只管开发，不管传播。开发出一款不错的工具之后，公众号上写的全是『怎么攻击』『拿下了什么资产』『绕过了什么防线』。这种写法在技术圈内或许显得硬核，但在外界眼中（尤其是监管与舆论场）会直接把作者的『出发点』暴露成『搞破坏』。这是合规上的致命伤。

正确的姿态是从攻击视角切换到防守视角：把自己设定为一名企业安全建设与防御人员，工具是为了发现更多风险而开发的，案例展示的是『如何帮助企业评估网络安全状况』。同样的技术内容，换一种叙事角度，对工具的法律与舆论定位完全不同。

5.6 开源真正的价值：抬高行业基线

回到最初的提问——开源到底是不是双刃剑？答案是『是』。但更重要的是怎么用这把剑。把它握紧、藏在身后，受伤的概率反而更高；把它放在阳光下，配齐用户画像、场景定义、合规资质、防守视角传播这四件套，再叠加免责声明、授权范围、禁用滥用三件套个人自保，开源工具就完成了从『可能被恶意利用』到『推动行业基线抬升』的角色转换。技术中立，但传播立场不中立——这是每一位开源安全工具开发者必须先想清楚再动手的命题。

5.7 本章小结

本章把开源安全工具的双刃剑效应展开成一份可操作的自检清单。工具中立，但作者的传播立场必须明确：把工具放在阳光下，配齐用户画像 + 场景定义 + 合规资质 + 防守视角传播这套阳谋四件套，公司型项目用资质堆护城河，个人型项目则用免责声明 + 授权范围 + 禁用滥用三件套划清责任边界。Yakit 早期员工的焦虑提醒我们恐惧来自于不被理解，菜刀类比则点明了工具中立的边界条件——必须摆在超市里。海外报道 CyberStrike AI 被用于扫飞塔漏洞这类事件提醒每一位作者：舆论起来后再去辩解已经太晚，前置防御重于事后澄清。常见误区是公众号上只写攻击案例，反而把出发点暴露为搞破坏，正确的写法是从攻击视角切换为防守视角。开源的真正价值，从来不是某个人的技术展示，而是把行业整体的攻防基线一起抬高——这是双刃剑能向阳的那一面。

6 工程师的新素养：审美、原理与技术加权时代

讨论行至最后一程，圆桌从工具、模型与开源逐步收束到一个最贴近读者切身利益的问题：当 AI 已经能写出大段可运行的代码，工程师本人究竟还要培养什么、警惕什么？四位嘉宾分别从『写代码不再是瓶颈』『模型与工程化的阶段论』『底层原理是驾驭 AI 的前提』『审美危机与加权效应』四个角度作答，串成一条从能力素养到职业天花板的完整论述。本章把这一段对话整理为面向读者的行动指引：在 AI 时代，真正稀缺的不是会写代码，而是知道什么样的代码值得被写、什么样的产物值得被采纳。

6.1 从『写代码』到『驾驭代码』：审美成为新内功

坤哥首先抛出一个在工程师群体中已经形成共识的判断：写代码本身已经不再是瓶颈。这里的『写代码』特指『写出一个语法正确、能够运行的代码』，这件事在 AI 辅助下变得几乎零成本。真正稀缺的是回答另一类没有标准答案的问题——什么样的技术是好的，什么样的架构是优秀的，什么样的功能设计是可扩展、可维护、面向未来的。

这套对好坏的判断标准，被工程师群体习惯性地称作**技术审美 (Technical Taste / Smell)**。它在不同语境下也叫『品味』『taste』『smell』，本质上都是一回事：判断架构、设计模式、抽象层次、扩展点优劣的内在尺度。在没有 AI 的旧时代，这套审美是用最笨的方法磨出来的——硬查文档、硬读源码、硬写硬调，靠一行行代码与一夜夜熬出来的肌肉记忆，沉淀为对设计模式好坏的天然敏感。



图 17：技术审美的旧路径：靠苦读源码与亲手编码养成对架构好坏的判断力¹⁷

坤哥提醒，进入 AI 时代后这套养成路径正在失灵：新一代工程师如果在接触技术之初就重度依赖 AI 输出，自身审美会缺少独立打磨的阶段，最终与 AI 同质化——AI 给出的就被默认是好的，AI 没给出的就被默认是坏的。一旦失去这一独立判断的支点，工程师对架构与设计的把关能力就会从根本上塌陷。他给出的具体建议非常朴素：看到一段你觉得『精妙』的设计时，不要急着划走，也不要因为复杂而退却，认认真真把它读完，把背后的取舍想透，这是新时代仍然有效的内功修炼方式。

¹⁷ 视频画面时间区间：01:05:55–01:06:05。

审美同质化陷阱

盲信 AI 最隐蔽的代价不是写出 bug，而是审美与模型趋同。一旦你的好坏判断完全外包给 AI，你会失去识别精妙设计与平庸设计的独立坐标系：AI 推给你的就是好的，AI 没给你的就是坏的。新一代工程师必须刻意保留一段『不依赖 AI 阅读源码、不依赖 AI 重构代码』的训练时间，把审美锚点重新打回自己的脑子里，而不是漂浮在模型权重里。

6.2 阶段论：短期模型为王，长期工程化为王

赵一凡顺着坤哥的话题，从模型、工具、工程化三者的相对重要性给出一个分阶段的判断，回应了上一章关于『模型 vs 工程化』讨论的延续问题。

短期视角，他认为模型能力仍是决定胜负的最主要因素。原因是当前各家模型尚未进入同质化时代：以推理深度、工具调用稳定性、长链路任务一致性这几个维度衡量，Claude 与国内主流模型之间存在肉眼可见的差距，在比赛这种长链路任务上表现尤其明显——同样的题目，换上 Claude 往往很快被拿下，换回某些国内模型则可能在中途因为推理或调度断链而失败。在这个阶段押注模型，是最直接的杠杆。

长期视角则相反。当模型能力逐渐趋同——这是行业普遍预期的方向——真正能拉开差距的就不再是模型本身，而是围绕模型搭建的系统化能力，也就是**工程化能力**：记忆、上下文压缩、工具兜底、Scaffold、可控沙箱、verification 流水线等。工具在这一阶段更多扮演辅助和补充。对绝大多数无法亲自训练一个比 Claude 更强大模型的开发者而言，能稳定差异化的着力点只剩工程化——把同一个底座模型用得更顺手、更稳定、更适合自己的业务，是普通开发者唯一可控的护城河。

主持人追问：那这是否意味着工程师『写代码的能力』会被弱化？赵一凡给出的回答是否定的。AI 写代码不会让程序员消失，而是会让程序员的身份从『手敲实现者』转变为『需求清晰化者 + 测试验证者』——你需要把意图描述得足够精确，AI 才能生成可用代码；你需要具备足够的测试和审查能力，才能判断 AI 输出是否可信。这仍然是『生产代码』的过程，只是劳动结构发生了迁移，编码、产品设计、测试三种身份在同一个工程师身上被重新组合。

6.3 驾驭 AI 的前提：理解神经网络、记忆与 Scaffold

齐涵从自身这一年多的亲身经历回应同一个问题，给出了非常具体的工作感受。一年多以前他对 AI 写的代码并不信任——他的工作习惯是希望每一行代码都清楚知道在做什么，AI 一改他就要逐行复核，反而比自己手写还低效。但随着这一年多模型能力的迭代，他切实感到 AI 写出的代码越来越可靠。

但这种『可靠』有一个前提：你必须知道 AI 适合做什么、不适合做什么。比如要给工具加一个新功能时，他会反复问自己几个问题：这个需求我能不能向 AI 说清楚？AI 是否真的理解了需求？这个需求里有哪些重要的边界与坑点是我必须事先标记的？AI 输出之后我打算用什么方法 check？这些问题决定了 AI 是放大器还是噪声源。

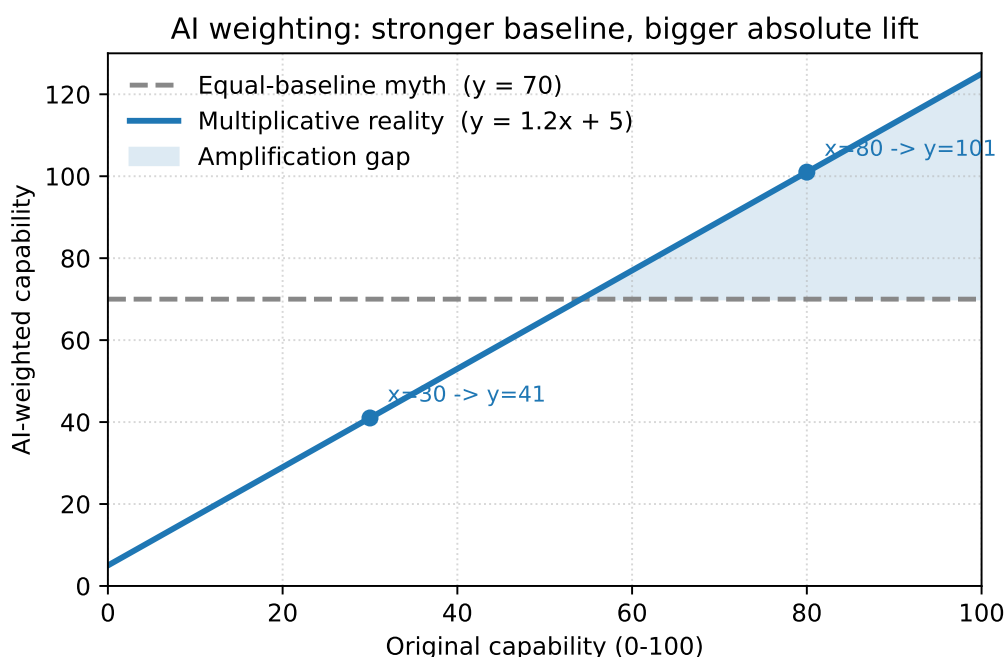


图 18: AI 平权幻觉与加权现实：原有水平越高，被放大的比例越大¹⁸

要培养这种能力，齐涵给出的根本路径是回到底层原理。他强调，工程师必须清醒地知道：AI 本质上只是一个大模型、一个神经网络；外围那些被频繁讨论的『记忆』『Scaffold』『MCP』，其实都不是 AI 时代才发明的新东西，它们是早已存在的工程概念在新场景下的组合。只要把这些底层原理弄懂——神经网络擅长什么、不擅长什么，记忆是怎么实现的，Scaffold 与 MCP 在 Agent 中扮演什么角色——后续无论模型如何快速演进，工程师都能找到合适的位置去驾驭它，而不是被它牵着走。换句话说，模型迭代越快，懂原理的人越占便宜，因为他们的判断锚点不会随模型版本变化而失效。

AI 时代工程师的三大新素养

综合坤哥、赵一凡、齐涵三位嘉宾的论述，AI 时代工程师真正需要补齐的不是『再多写一些代码』，而是以下三项内功：

- **技术审美**：对架构、可扩展性、设计模式优劣的独立判断力，是把关 AI 输出的最后一道防线；
- **底层原理理解**：把 AI 还原为神经网络、记忆、外挂式 Agent 框架、MCP 等可分析的工程组件，知道它擅长什么、何处会失效；
- **高精仪器驾驭能力**：把 AI 视为一台参数繁多的精密仪器而非黑箱按钮，掌握调参、调度、验证全链路的工程化方法。

三者缺一，AI 都更像一个不可控的放大噪声源，而非真正的生产力杠杆。

6.4 审美危机与加权效应：AI 不是平权而是放大器

李鑫在嘉宾轮最后做了两点补充，把前面三位的话题进一步推向了全行业的警示。

¹⁸ 本图为概念示意图，非视频原画。

第一点关于审美。他坦言之前听到『AI 时代审美很重要』时并不太走心，直到亲眼看到周围同学用 AI 做的海报、生成的素材，发现『有点丑』『有点别扭』，才意识到审美在 AI workflow 里是真正的决定性变量。同样的现象在前端开发中更为明显——AI 写前端这件事在工程上已经非常便捷，但如果使用者本身审美不行，他给出的提示词本身就不到位，AI 沿着这条提示词路径生成的页面也只会延续这种审丑。审美不是装饰品，它是 prompt 上游、是验收下游、是整个 AI 生产管线里那把唯一握在人手上的尺子。

第二点是对『AI 让一切变简单』这一直觉的彻底反驳，也是本章最重要的一个判断。市面上常见的说法是『技术平权』『人人都能写代码』『以后越来越容易』，但李鑫看到的现实恰好相反：所谓平权只是一种幻觉，AI 实际带来的是**技术加权而非平权 (Capability Amplification)**。AI 对每个人能力的加持是按比例放大的——本来很厉害的人，加持之后变得更厉害；本来水平相对扁平、相对平庸的人，加持的天花板也被原有水平限制住，无法通过 AI 一夜抹平。AI 给一个人技术能力的天花板提升，与这个人原有的能力和认知是强相关的，而不是把所有人按同一基准线托起。

加权而非平权：用比例理解 AI 的真实效应

所谓『加权』可以这样直观理解：假设把一名工程师的综合能力归一化到 0-100 的区间，AI 不是给每个人加上同一个固定加分，而是按比例放大原有水平。

- 原本 80 分的工程师，被 AI 加权后可能跃升到 95 分——他能精确描述需求、识别边界、验收输出，AI 的乘数效应被吃满；
- 原本 40 分的工程师，被 AI 加权后大约停留在 50 分——他既写不出准确的提示词，也无力验收 AI 产物，乘数效应被自身天花板压住。

结论看似反直觉但完全成立：AI 时代群体的能力差距非但没有被抹平，反而被进一步拉开。能不能被 AI 加权得多，最终取决于人本身的能力底子。

6.5 高精仪器隐喻：剩给人类的都是更难骨头

加权效应进一步延伸出对人才要求的判断。李鑫指出，模型越强、Agent 越强、AI 技术发展越快，对未来人才驾驭能力的要求其实是越高，而不是越低。原因很朴素：当你把那些重复枯燥的事情整体丢给 AI 之后，你剩下要亲自处理的事情，必然是 AI 不容易搞定的事情；而 AI 不容易搞定的事情，难度天然就不会低。



图 19: 高精仪器隐喻：AI 自动化能力强，但要用好它，需要驾驭的参数与按钮也更复杂¹⁹

为此他给出了贯穿本章最后一节的核心比喻：AI 是一台**高精精密仪器**。它的自动化能力很强，但要把它真正用好，可调的参数、可拨的按钮、需要协调的子系统极其丰富。具体落到工程师视角可以从三个维度感受这种复杂度的上升：

1. **可调参数变多**：从过去的几行命令行参数，扩展到 Prompt、工具列表、记忆策略、采样参数、上下文压缩策略、调度范式等数十个互相耦合的旋钮；
2. **调度复杂度变高**：单步调用变成多 Agent 协作链路，每一步都涉及 ReAct/Plan-Execute/假设-探索等不同范式的取舍与回溯；
3. **容错空间变窄**：模型幻觉、算力波动、工具失败、上下文溢出等不确定性叠加在一起，留给人的容错窗口反而比传统工程更紧。

过去会写一个 Python 脚本、会用一个扫描器，可能就足以胜任一份初级工作；今天要驾驭这样一个『庞然大物』，工程师需要掌握的东西成倍增加。这一比喻彻底打破了『AI 让我以后只要描述需求就行』的简单设想——剩给人的工作不仅没变少，平均难度反而被 AI 抬了一个台阶。

主持人最后做了一个简短而有力的收束：AI 工具人人都能用，整个人类的能力基线都会被抬高，因此最终决定胜负的不再是『谁会用 AI』，而是『谁被 AI 加权得更多』。这与坤哥的审美论、赵一凡的工程化论、齐涵的原理论在结论上完全一致——AI 时代不是『躺平时代』，而是『加权时代』。

6.6 总决赛预告与议题延续

在嘉宾轮结束后，主持人对今天的圆桌做了一个简短总结：今天主要从技术演进的视角讨论了 AI 与工具的关系，也讨论了智能渗透在当前阶段所展现出的人力边界。在线上四日的赛程之后，比赛还将延续到线下：

- **时间地点**：4 月 25 日，北京线下总决赛与 AI 攻防沙龙；
- **上午议程**：前十战队现场答辩；
- **下午议程**：多位专家围绕智能渗透、NG 安全、工具智能化方向做主题分享与圆桌论坛；

¹⁹ 视频画面时间区间：01:14:55-01:15:05。

- **现场环节：**技术与 HR 对接，把比赛能力沉淀延伸到职业生态。

主持人特别提示，明晚 20 点的最后一场直播将做整体比赛总结，回顾本届比赛的亮点与趋势——这意味着本讲义所讨论的『工具智能化』『模型 vs 工程化』『真实攻防的最后一公里』『开源合规』『工程师新素养』等议题不会随圆桌结束而中断，而会在线下与最终一场直播中延续展开。

6.7 本章小结

把全章观点合在一起，可以得到一份贴近读者本人的行动清单：

- **写代码不再是瓶颈，但驾驭代码的审美是：**不要把对架构与设计好坏的判断权外包给 AI，刻意保留独立阅读与重构源码的训练；
- **阶段论决定投入比例：**短期内模型差异仍是决定性变量，长期模型趋同后，工程化能力（记忆、Scaffold、MCP、verification）会接管胜负手；
- **原理是驾驭的前提：**把 AI 还原为神经网络 + 外挂式 Agent 框架的组合，知道每个组件擅长什么，才能在模型迭代中保持判断稳定；
- **警惕平权幻觉：**AI 是按比例放大的加权器，不是把所有人推到同一水平的平权器；自身能力天花板没抬高的部分，AI 也救不了；
- **接受高精仪器的复杂性：**剩给人的工作只会更难，可调参数、调度复杂度、容错空间都对工程师的综合素养提出更高要求；
- **把比赛延伸到线下：**4 月 25 日北京总决赛是本届议题的延续点，也是个人审美、原理解与驾驭能力进入实战交流的下一站。

至此，本章完成了讲义的『行动指引』承重墙：从 AI 与工具的范式重塑，到模型与工程化的力量分配，再到工程师本人在这场变革中应当成为怎样的人——AI 没有让从业者变得不重要，反而把对从业者的要求悄悄抬高了一个量级。剩下的一切，都取决于读者愿不愿意把自己也升级为一台『高精密仪器』。

7 总结与延伸：把 AI 与工具放进同一台机器

上一章我们把全部讨论收束到了工程师本人——审美、原理、驾驭力。本章把六章打通，回到一个更高的视角：把整场圆桌作为一个系统看，AI 与工具究竟应该被放进同一台怎样的机器？整场圆桌从『第二届智能渗透挑战赛第四日战况』的小切口，拉出了 AI 时代安全工具一系列结构性变化：工具不死、模型决定上限、工程化决定下限、CTF 与真实攻防之间还存在最后一公里、开源工具必须站在阳光下、工程师的素养重心从『写代码』迁移到『驾驭代码』。本节先回放嘉宾在结尾段落里那些经常被互动的客套盖住、却真正承担教学价值的实质收束，然后给出我们对全场逻辑的二次压缩。

7.1 嘉宾结尾的实质性收束

去掉互相致谢、礼貌寒暄之后，嘉宾在结尾真正传达的教学要点可以归并为如下六条。它们并不是新观点，而是把前面六章里被分散提到的论点显式化、相互勾连。

- AI 时代的工具不是被削弱，而是被重定义为提供给 AI 的『确定性能力接口』；底层能力工具反而比胶水脚本更不可替代。
- 同一个模型在不同团队手里口碑两极，差异主要由 Agent 的工程化能力决定：模型决定上限，工具兜底与外挂式 Scaffold 决定下限。
- 比赛靶场是规范化的沙盒，真实攻防却是琐碎、碎片化、需要协作的；接下来要靠多智能体协同与更完备的 Benchmark 弥合这条最后一公里。
- 开源安全工具必须用『阳谋四件套』走在阳光下：用户画像 + 场景定义 + 合规资质 + 防守视角传播，事后辩解永远赶不上前置防御。
- 工程师必须培养技术审美、底层原理认知与高精仪器驾驭能力；AI 不是平权而是按个人水平等比放大产出。
- 4 月 25 日北京线下总决赛将延续工具智能化主题，举办十强答辩、专家分享与 HR 对接，欢迎到场或线上观摩，把今天的讨论延伸进下一个阶段。

7.2 把全场压缩成一套可记的判断框架

如果用一句话压缩本讲义的核心论断：**Agent 的真实战力 = 模型上限 × 工程化下限 × 场景适配度 × 合规可持续性 × 工程师驾驭力**。前三个量决定它能做什么，第四个量决定它能不能做下去，最后一个量决定谁才是这台机器真正的主人。

讲义五条主干结论

1. **工具被重新定义**。工具是给 AI 用的确定性能力接口，对稳定性、原子化粒度、协议规范化的要求不降反升；胶水脚本退场，底层能力被强化。
2. **上限由模型决定，下限由工程化决定**。选模型决定天花板，搭 Scaffold（记忆/Cursor/沙箱/MCP/verification）决定地板，工具兜底把不确定性空间压成可控的小集合。
3. **原子化 + 意图感知 + 工具兜底，是把超大工具池变得可调度的三件套**。不要一次塞所有工具进上下文，要按 30 秒粒度动态推荐；工具先打 50 分，AI 再加分。
4. **CTF 不等于真实攻防**。多智能体协同（社工 / 爆破 / 验证码 / 流量分析）+ 更贴近碎片化场景的 Benchmark，是把『会解题』升级为『能干活』的必经之路。
5. **阳谋路线 + 加权人才**。开源工具用用户画像、合规资质、防守视角传播构成合规闭环；工程师用审美、原理、驾驭力获得 AI 的等比放大效应，强者愈强。

7.3 跨章串联：六章的因果脉络

讲义六章并不是六个并列话题，而是顺着同一根因果链一层层往里走。**第一章**给出现象（智能渗透挑战赛把 AI 与工具放在同一个赛场上验证）；**第二章**回答了第一个直觉性问题（AI 时代工具会被淘汰吗）——工具不死，但角色彻底变了；**第三章**回答了第二个反直觉问题（同一模型为什么口碑两极）——工程化才是真正的变量，并把『确定性 vs 不确定性』的分工原则显式化；**第四章**把视野从赛场拉回真实战场，承认现在的 AI Agent 在碎片化场景下还差最后一公里；**第五章**把镜头转向工具开发者本身的合规与传播命题，回答『为什么开源安全工具不能藏着掖着』；**第六章**把所有讨论收束到从业者层面，给出在 AI 加权时代如何成为『被加权幅度更大的那一档人』的具体路径。

把它再压一次，就是一句话：**从赛场到工具，从工具到分工，从分工到边界，从边界到合规，从合规到工程师**——这是一条由现象到机制、由机制到行动指引的完整教学链条。

7.4 延伸：仍未解决的开放问题

整场讨论清楚地承认还有许多问题没有答案；对读者而言，这些恰好是接下来值得投入的下一站。

讲义留下的五个开放问题

1. 如何构建可量化考察 Agent 真实攻防能力的下一代 Benchmark？需要把『碎片化场景』转化为可评判、可对比、可复现的题目结构。
2. 多智能体协同（社工 / 爆破 / 验证码 / 流量 Agent）的调度协议与共享记忆该如何设计？是平面合议还是层级 Plan-Execute？
3. 在国密、私有协议、专属设备等长尾场景，工具与模型应如何分担稳定性与覆盖度？哪一段的『工具兜底』最划算？
4. 开源安全工具的合规背书在跨境舆论场景下如何前置？是不是需要类似 SBOM 的『工具用户与场景披露表』？
5. 工程师如何系统性培养技术审美而不被 AI 同质化？是不是必须坚持读源码、做苦活、写有偏好的工程决策？

7.5 读者行动清单

最后给出一份带优先级的行动清单。它脱胎自全场嘉宾的具体建议，但被我们重新整理为可执行步骤，方便读者在真实工作中立刻落地。

1. **清点你的工具栈。**哪些是胶水脚本？哪些是底层能力？把胶水部分留给 AI 自动生成，把底层能力做扎实、提供稳定 MCP 接口。
2. **为你的 Agent 定义『地板』。**用工具兜底先把 50 分稳态拿下，把 verification 和工具回滚机制写进流程，再让 AI 在剩余空间里加分。
3. **用意图感知重构工具调度。**不要把全部工具塞进上下文，按任务窗口动态推荐；明确每个工具的『能力描述 + 输入输出 + 错误模式』元数据。
4. **把开源放在阳光下。**准备一份用户画像、场景定义、免责声明、合规资质打包文档，公众号传播改用防守视角，杜绝纯攻击叙事。
5. **每周练一次审美。**选一份 AI 生成的代码或文档，强迫自己用『重构者』而非『接受者』视角批判它；同时坚持读 1-2 段优秀开源项目的源码。
6. **给自己建一台高精仪器。**把神经网络基础、记忆/Scaffold/MCP 原理、典型 Agent 框架（CyberStrike AI / Yakit-AI / Cursor / Claude Code）摸熟，让你的『驾驭力』追得上 AI 的更新速度。

7.6 致谢与延续

感谢腾讯安全众测组织本场圆桌，感谢主持人段刚以及四位嘉宾——赵一凡（CyberStrike AI 作者）、坤哥（万界安全 CDO，Yakit 项目负责人）、齐涵（平台开发）、李鑫（云鼎实验室攻防负责人）——把一场原本可以变成『复盘比赛战况』的活动开到了这种深度。讲义的不足之处由整理者承担；要更完整地获取嘉宾的现场表达与未录入讲义的细节，请参考视频原片：

<https://www.bilibili.com/video/BV175ofBTEpR>

下一站是 4 月 25 日北京线下总决赛与十强答辩。把这份讲义里的任意一条结论带进现场，回看现场嘉宾如何用实战检验它，你会发现『工具、模型、工程化』三者的关系还会继续被重塑。